PRACTICAL 4-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 4 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Password Checker, Chat Filter, Template Filler and Part B: Validators, Command Parser, Statement Formatter.

CODE SCREENSHOT:

ChatFilter.java 1

Part A > Practical 4-A > ChatFilter.java > ...

```java
import java.util.*;

public class ChatFilter {
    Run | Debug
    public static void main(String[] args) {
        String[] logs = { "10:05 alice Hello there", "10:06 bob How are you", "malformed", "10:07 cara HELLO again" };
        Scanner sc = new Scanner(System.in);
        System.out.print("Keyword: ");
        String key = sc.nextLine().toLowerCase();
        StringBuilder report = new StringBuilder();
        int matches = 0;
        for (String line : logs) {
            String[] parts = line.split(" ", 3);
            if (parts.length < 3)
                continue;
            if (parts[2].toLowerCase().contains(key)) {
                matches++;
                report.append(parts[0]).append(" ").append(parts[1]).append(": ").append(parts[2])
                        .append(System.lineSeparator());
            }
        }
        System.out.println("Matches: " + matches);
        System.out.print(report);
    }
}

```

Code Screenshot 1

PasswordChecker.java

Part A > Practical 4-A > PasswordChecker.java > ...

```java
public class PasswordChecker {
    public static String strength(String pw) {
        int score = 0;
        if (pw.length() >= 8)
            score++;
        if (pw.matches(".*[A-Z].*"))
            score++;
        if (pw.matches(".*\\d.*"))
            score++;
        if (pw.matches(".*[^A-Za-z0-9].*"))
            score++;
        return score <= 1 ? "Weak" : score <= 3 ? "Medium" : "Strong";
    }

    Run | Debug
    public static void main(String[] args) {
        for (String p : new String[] { "abc", "Abcd1234!", "hello123", "HELLO!!" })
            System.out.println(p + " -> " + strength(p));
    }
}

```

Code Screenshot 2

TemplateFiller.java

Part A > Practical 4-A > TemplateFiller.java > ...

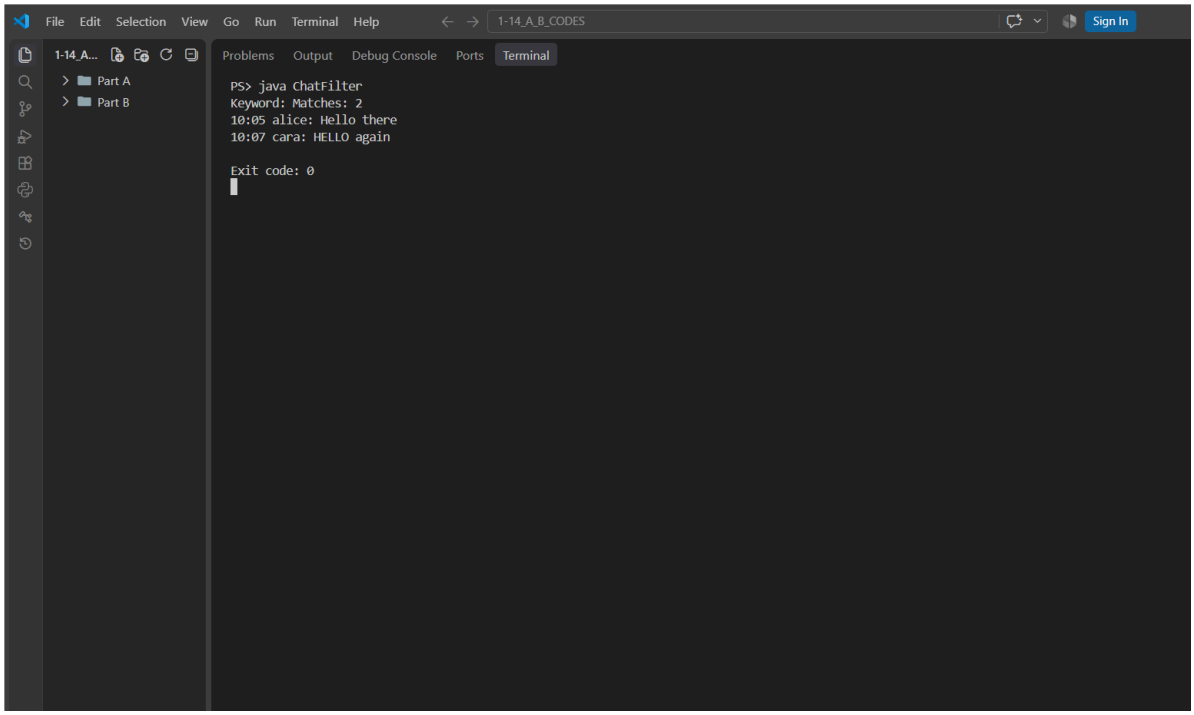
```java
import java.util.regex.*;

public class TemplateFiller {
    Run | Debug
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";
        String[] names = { "name", "id" }, values = { "Riya", "A07" };
        Matcher m = Pattern.compile("\\{(\\w+)\\}").matcher(template);
        StringBuilder out = new StringBuilder();
        int last = 0;
        while (m.find()) {
            out.append(template, last, m.start());
            String key = m.group(1), value = "[?]";
            for (int i = 0; i < names.length; i++)
                if (names[i].equals(key))
                    value = values[i];
            out.append(value);
            last = m.end();
        }
        out.append(template.substring(last));
        System.out.println(out);
    }
}

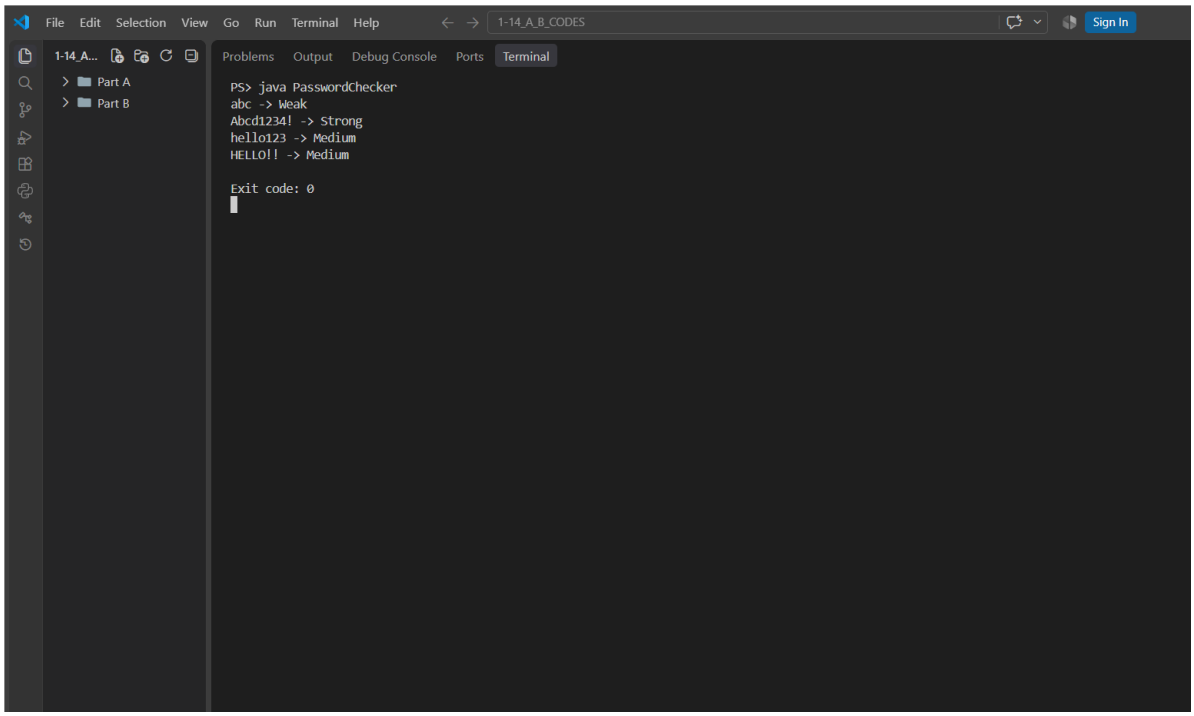
```

Code Screenshot 3

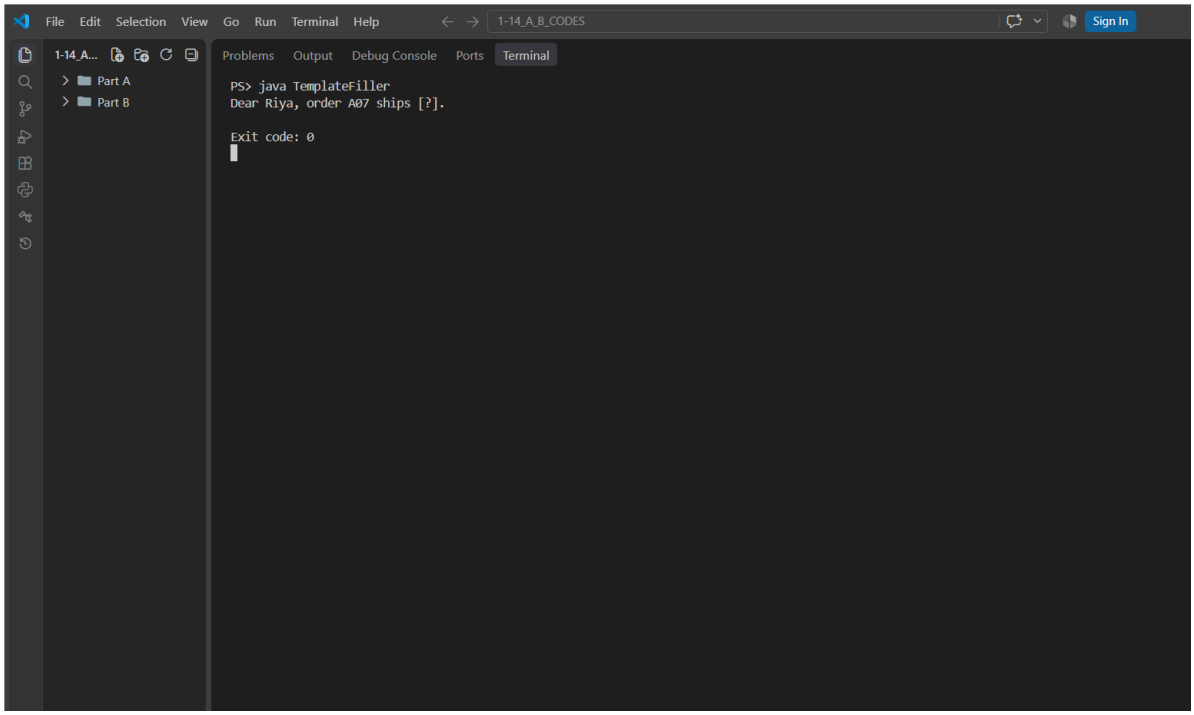
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Validate and parse text using String methods, StringBuilder, regular expressions, and reusable helper classes.